Data Structures and Algorithms – IT1170

Practical 3 – Stacks / Answers

Exercise 1

1. Create a class StackArray that implements a stack using an array and stores integers.

2. Implement the following methods:

   → push(int item): Adds an item to the stack.
   → pop(): Removes and returns the top item.
   → peek(): Returns the top item without removing it.
   → isEmpty(): Checks if the stack is empty.
   → isFull(): Checks if the stack is full.

3. Introduce a new method named display() to print the elements in the stack.

4. Write a main method to test the stack operations.

- Q1, Q2, Q3 - StackArray Class

```
class StackArray {

    private int[] stack;
    private int top;
    private int capacity;

    public StackArray(int size) {
        capacity = size;
        stack = new int[capacity];
        top = -1;
    }

    public boolean isEmpty() {
        return top == -1;
    }

    public boolean isFull() {
        return top == capacity - 1;
    }

    public void push(int item) {

        if (isFull()) {
            System.out.println("Stack Overflow");
```

```
            return;
        }

        stack[++top] = item;
    }

    public int pop() {

        if (isEmpty()) {
            System.out.println("Stack Underflow");
            return -1;
        }

        return stack[top--];
    }

    public int peek() {

        if (isEmpty()) {
            return -1;
        }

        return stack[top];
    }

    public void display() {

        if (isEmpty()) {
            System.out.println("Stack is Empty");
            return;
        }

        System.out.print("Stack Elements: ");

        for (int i = top; i >= 0; i--) {
            System.out.print(stack[i] + " ");
        }

        System.out.println();
    }
}
```

- Q4 - Main Method

```
public class Main {

    public static void main(String[] args) {

        StackArray stack = new StackArray(5);

        stack.push(10);
        stack.push(20);
        stack.push(30);

        stack.display();

        System.out.println("Top Element: "
                + stack.peek());

        System.out.println("Removed: "
                + stack.pop());

        stack.display();
    }
}
```

Exercise 2

5. Modify the above program to take a string input. Your program needs to take the input from the user and print the reversed user input. Hint: You may have to change the stack to store characters.

```
import java.util.Scanner;

class CharStack {

    private char[] stack;

    private int top;

    public CharStack(int size) {

        stack = new char[size];

        top = -1;

    }

    public void push(char ch) {

        stack[++top] = ch;
```

```
    }
    public char pop() {
        return stack[top--];
    }
    public boolean isEmpty() {
        return top == -1;
    }
}
public class ReverseString {
    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.print("Enter String: ");
        String str = input.nextLine();
        CharStack stack =
                new CharStack(str.length());
        for(int i=0;i<str.length();i++) {
            stack.push(str.charAt(i));
        }
        System.out.print("Reversed String: ");

        while(!stack.isEmpty()) {
            System.out.print(stack.pop());
        }
    }
}
```

Exercise 3

6. Assume the above StackArray class. You need to introduce a new method to this class to check whether parentheses are balanced. Write Java code segment for boolean isBalanced(String str) to take a string input from the user. Using the stack data structure your program should return true for balanced parentheses i.e. {[()]}, and false for imbalanced parentheses i.e. {{([])}].

```
import java.util.Stack;
public class ParenthesesCheck {
    public static boolean isBalanced(String str) {
        Stack<Character> stack =
                new Stack<>();
        for(char ch : str.toCharArray()) {
            if(ch == '(' ||
               ch == '{' ||
               ch == '[') {
                stack.push(ch);
            }
            else if(ch == ')' ||
                    ch == '}' ||
                    ch == ']') {
                if(stack.isEmpty()) {
                    return false;
                }
                char top = stack.pop();
                if((ch == ')' && top != '(') ||
                   (ch == '}' && top != '{') ||
                   (ch == ']' && top != '[')) {
                    return false;
                }
```

```
            }
        }
        return stack.isEmpty();
    }
    public static void main(String[] args) {
        System.out.println(
                isBalanced("{[()]}"));
        System.out.println(
                isBalanced("{{([])}]"));
    }
}
```